

Java OOP — Class & Object

Beginner-friendly notes for Java placement preparation

1. What is OOP?

OOP = Object-Oriented Programming. Java is mainly an object-oriented programming language. OOP lets us model real-world things as objects.

Example: A Student can have name, age and marks as properties, and study(), writeExam() and displayDetails() as behaviors.

2. What is a Class?

A **class is a blueprint/template for creating objects**. Think of a house blueprint: the blueprint is not the actual house. Similarly, a class defines the structure from which objects are created.

```
class Student {
    String name;
    int age;

    void study() {
        System.out.println("Student is studying");
    }
}
```

3. What is an Object?

An **object is an actual instance of a class**. The following statement creates a Student object:

```
Student s1 = new Student();
```

Student → class/type **s1** → reference variable **new Student()** → creates the object

4. Basic Syntax

```
class ClassName {
    // variables
    // methods
}

public class Main {
    public static void main(String[] args) {
        ClassName object = new ClassName();
    }
}
```

5. First Example

```
class Student {

    String name;
    int age;

    void display() {
        System.out.println(name);
        System.out.println(age);
    }
}

public class Main {
```

```

    public static void main(String[] args) {

        Student s1 = new Student();

        s1.name = "Sanjay";
        s1.age = 21;

        s1.display();
    }
}

```

Output: Sanjay

21

6. Understand Student s1 = new Student() Carefully

```

Student s1 = new Student();
  ↓      ↓      ↓
class  ref  object

```

Student is the type. **s1** is the reference variable. **new** creates a new object. **Student()** invokes the constructor.

7. Accessing Object Variables

Use the **dot operator (.)** to access an object's members:

```

s1.name = "Rahul";
s1.age = 20;

s1.display();

```

General pattern: **object.variable** and **object.method()**.

8. Multiple Objects

```

class Student {

    String name;
    int age;

    void display() {
        System.out.println(name + " " + age);
    }
}

public class Main {
    public static void main(String[] args) {

        Student s1 = new Student();
        Student s2 = new Student();

        s1.name = "Sanjay";
        s1.age = 21;

        s2.name = "Rahul";
        s2.age = 22;

        s1.display();
        s2.display();
    }
}

```

Output:

Sanjay 21

Rahul 22

Both objects have the same structure but store different data.

9. Why Do We Need Classes and Objects?

Suppose you need information for 100 students. A Student class lets you create many Student objects, with each object maintaining its own data. This makes programs easier to organize and model.

10. Class vs Object

Class	Object
Blueprint/template	Actual instance
Defines structure	Represents a particular entity
Used to create objects	Created from a class
Example: Student	Example: s1

11. Real-World Example — Bank Account

```
class BankAccount {  
  
    String name;  
    double balance;  
  
    void displayBalance() {  
        System.out.println(balance);  
    }  
}  
  
BankAccount account1 = new BankAccount();  
BankAccount account2 = new BankAccount();  
  
account1.name = "Sanjay";  
account1.balance = 50000;  
  
account2.name = "Rahul";  
account2.balance = 30000;
```

12. Important Interview Question

Is a class an object? No. A class is a blueprint/template, while an object is an instance created from that class.

```
class Car {  
}  
  
Car c1 = new Car();  
  
Car → class  
c1 → reference to an object
```

13. Basic Memory Concept

```
Stack                                Heap  
  
s1 -----> Student Object
```

name
age

For now, remember the key point: **new creates an object.**

14. Logic-Building Example

Problem: Create a Student class that stores name and marks and displays whether the student passed or failed.

Step 1: Identify the entity → Student

Step 2: Identify properties → name, marks

Step 3: Identify behavior → checkResult()

Step 4: Create an object → Student s1

Step 5: Assign values

Step 6: Call the method

```
import java.util.*;

class Student {

    String name;
    int marks;

    void checkResult() {

        if (marks >= 40) {
            System.out.println(name + " Passed");
        } else {
            System.out.println(name + " Failed");
        }
    }
}

public class Main {

    public static void main(String[] args) {

        Scanner sc = new Scanner(System.in);

        Student s1 = new Student();

        s1.name = sc.nextLine();
        s1.marks = sc.nextInt();

        s1.checkResult();
    }
}
```

Input: Sanjay / 75

Output: Sanjay Passed

15. Important Pattern to Remember

1. Identify the entity
↓
2. Create a class
↓
3. Identify its properties
↓
4. Create variables
↓
5. Identify its actions
↓
6. Create methods
↓
7. Create object
↓

8. Assign values



9. Call methods

Example: Employee → class Employee → properties name/salary/age → methods display()/calculateSalary() → object Employee e1.

16. Practice Problem

Create a class called **Car**. It should contain **brand** and **price**. Create a method **display()** that displays the car details. In **main()**, create one Car object, read brand and price using Scanner, store them in the object, and call **display()**.

Next OOP Topic: Constructors — why we can write `Car c1 = new Car("BMW", 5000000);` instead of assigning each field separately.